

3Boxes HRMS

Updated Architecture Technical Document

Document	Architecture & Technical Specification
Version	2.0
Date	September 2026
Status	Production — Golden Rule Enforced
Scope	Multi-Tenant DB Isolation, RBAC, Probation Workflow

Table of Contents

Section	Title	Page
1	Golden Rules & Database Separation	3
2	Multi-Tenant Database Architecture	4
3	Database Routing Mechanism	5
4	Role-Based Access Control (RBAC)	6
5	Employee Status & Type	7
6	Probation Workflow	8
7	API Route Security & Schema Sync	9
8	Official Email Validation	10
9	Super Admin Capabilities	10
10	Confirmation & Compliance Checklist	11

1. Golden Rules & Database Separation

The 3Boxes HRMS platform operates on a strict multi-tenant architecture where each tenant (organization) has its own dedicated database. This ensures complete data isolation, security, and scalability. The following Golden Rules are permanently enforced and cannot be overridden:

GOLDEN RULE: The Super Admin (platform) database and Tenant databases MUST be separate. The platform DB stores ONLY tenant registrations, user accounts (super_admin), and analytics. Tenant data (employees, companies, payroll, etc.) is NEVER stored in or read from the platform DB.

1.1 Database Separation Rules

Rule	Description	Status
Rule 1	Each link has a SEPARATE database	ENFORCED
Rule 2	Platform DB stores ONLY: Tenant, TenantDatabase, User (super_admin), TrialRegistration, SubscriptionPlan	ENFORCED
Rule 3	Tenant DBs store: Company, Department, Employee, Payroll, Attendance, Leave, etc.	ENFORCED
Rule 4	NO tenant data is read from or written to the platform DB	ENFORCED
Rule 5	Super admin fetches tenant data via getDbForTenant(slug) when viewing a specific tenant	ENFORCED
Rule 6	Official email only for login (no personal emails like gmail/yahoo)	ENFORCED
Rule 7	No dummy/sample data on the LIVE site (3boxeshrms.com)	ENFORCED

2. Multi-Tenant Database Architecture

The platform uses Neon PostgreSQL with dedicated databases for each tenant. The connection routing is handled by the tenant-db.ts module, which resolves the correct database based on the request hostname, JWT token, or tenant slug.

2.1 Database Map

Link	Database	Purpose
3boxeshrms.com	neondb (Platform DB)	Super Admin: Tenant registrations, user accounts, analytics
marqaitechgroup.3boxeshrms.com	tenant_marqaitechgroup	MarqAI tenant: Companies, employees, payroll, etc.
nexus-hrms-mu.vercel.app	tenant_demo	Demo link: Sample data for showcase
[tenant].3boxeshrms.com	tenant_[slug]	Future tenants: Each gets own dedicated DB

2.2 Platform DB Contents (neondb)

Table	Purpose
Tenant	Tenant registrations: name, slug, domain, plan, status
TenantDatabase	Maps each tenant to its dedicated DB connection string
User	Platform-level users (super_admin only)
TrialRegistration	Trial signup requests
SubscriptionPlan	Pricing plans and features

2.3 Tenant DB Contents (tenant_[slug])

Table	Purpose
User	Tenant users (tenant_admin, HR admin, employees)
CompanyGroup	Company groups within the tenant
Company	Companies (e.g., MARQ AI TECH, 3 BOXES LUXURY)
Branch	Company branches/locations
Department	Departments within companies
Designation	Job titles and levels
Employee	Employee records with all fields
PerformanceReview	Probation reviews and performance data
LeaveType, LeaveBalance, LeaveRequest	Leave management
Attendance, Shift, Holiday	Attendance tracking
Payroll, PayrollRun, PayrollInput	Payroll processing
Notification, AuditLog, LoginActivity	System logs

3. Database Routing Mechanism

The `getDb(request)` function in `src/lib/tenant-db.ts` is the PRIMARY function all API routes use to determine which database to connect to. It resolves the tenant context through a priority chain:

3.1 Resolution Priority

Priority	Source	Example
1	x-tenant-slug header (set by middleware from subdomain)	onap-itechgroup
2	Demo domain detection from host header	nexus-hrms-mu.vercel.app
3	JWT token tenantId (decoded, then slug looked up)	From JWT payload
4	?tenantId= query param (super admin company switcher)	?tenantId=abc123
Fallback	Platform DB (for super_admin with no tenant context)	platformdb

3.2 getDbForTenant(slug) Function

Once the tenant slug is resolved, `getDbForTenant(slug)` looks up the `TenantDatabase` record in the platform DB to find the connection string for the tenant's dedicated database. It creates a `PrismaClient` connected to that database and caches it for subsequent requests.

If no `TenantDatabase` record exists, it falls back to the platform DB (shared mode with `tenantId` filtering). This fallback is used ONLY during tenant onboarding before the dedicated DB is provisioned.

Type 1 (CORRECT): Super admin uses `getPlatformDb()` to look up which tenant a company belongs to (Company -> Group -> Tenant -> `getDbForTenant()`), then fetches actual data from the tenant's dedicated DB.

Type 2 (REMOVED): Falling back to platform DB for employee/company data queries. This was removed in the latest update to enforce the Golden Rule.

4. Role-Based Access Control (RBAC)

The RBAC system defines 5 standard roles with hierarchical access levels. Each role has a data scope that determines how much data they can see across all modules. Legacy roles (`company_hr_admin`, `hr_admin`, etc.) are mapped to these standard roles via the `LEGACY_ROLE_MAP`.

4.1 Standard Roles

Role	Level	Data Scope	Description
<code>super_admin</code>	0	all	Platform-level full access across all tenants
<code>tenant_admin</code>	1	all	Tenant-level full access within their tenant
<code>admin</code>	2	all	Company-level admin (HR, Finance, IT combined)
<code>manager</code>	3	team	People manager: own + direct reports
<code>employee</code>	4	self	Self-service: own records only

4.2 Legacy Role Mapping

Existing users created with legacy role names are automatically mapped to the new 5-role system:

Legacy Role	Mapped To	Data Scope
<code>company_hr_admin</code>	<code>admin</code>	all
<code>hr_admin</code>	<code>admin</code>	all
<code>finance_admin</code>	<code>admin</code>	all
<code>it_admin</code>	<code>admin</code>	all
<code>manager</code>	<code>manager</code>	team
<code>employee</code>	<code>employee</code>	self
<code>recruiter</code>	<code>admin</code>	all
<code>candidate</code>	<code>employee</code>	self

4.3 Module Access

Each module in the sidebar is tagged with roles that can access it. Employees see only self-service options (My Profile, Apply Leave, Apply Regularization, My Payslips). Managers see self-service + team management. Admins see everything. The filtering is done in `Sidebar.tsx`, `layout.tsx`, `ModuleStrip.tsx`, and `modules/page.tsx` using both `moduleKey` and `per-item` roles arrays.

5. Employee Status & Type

5.1 Employee Status

The Employee Status field determines the current employment status and drives the probation workflow:

Status	Description	Probation Workflow
Confirmed	Employee has completed probation	No auto-creation
Probation	Employee is in probation period	Auto-creates probation review
Contract	Contract employee	No auto-creation
Internship	Intern	No auto-creation
Consultant	External consultant	No auto-creation

5.2 Employee Type

The Employee Type field determines the employment contract type and will be connected to policy configuration fields across all modules:

Type	Description
Full Time	Full-time permanent employee
Part Time	Part-time employee
Contract	Fixed-term contract employee
Temporary	Temporary/seasonal worker
Consultant	External consultant

Probation Auto-Creation: When an employee is created or edited with status "Probation", the system automatically creates a PerformanceReview (reviewCycle="Probation") with:

- probationStartDate = Date of Joining
- probationEndDate = Editable (default: joining + 6 months)
- status = "pending" (goes to HR/Admin for review)
- workflowStage = "hr_review"

6. Probation Workflow

The probation workflow is a 2-level approval process: HR review followed by MD/Admin final confirmation. When an employee's status is set to "Probation", a review record is automatically created and routed through this workflow.

6.1 Workflow Stages

Stage	Who	Action	Result
hr_review	HR/Admin (admin, company_hr)	Submit decision to manager / Move to Rejected	MD/Admin review; MD/Admin notified
md_review	MD/Admin (super_admin, tenant_admin)	Final decision: Confirm / Extend / Reject	Employee status updated; HR notified
completed	System	Workflow complete	Employee status = "confirmed" if approved

6.2 Alert System (Cron Endpoint)

A daily cron job endpoint (/api/cron/probation-alerts) checks all pending probation reviews and sends in-app notifications to HR/Admin at:

Timing	Alert Type	Notification
10 days before end date	Initial alert	Probation Review Due in 10 Days
3 days before end date	Urgent alert	URGENT: Probation Review Due in 3 Days
1 day before end date	Critical alert	CRITICAL: Probation Review Due Tomorrow
On end date	Overdue alert	OVERDUE: Probation Period Ended

Cron Setup: Set up a daily cron job to hit:
[https://\[domain\]/api/cron/probation-alerts?secret=3boxes-hrms-cron-2026](https://[domain]/api/cron/probation-alerts?secret=3boxes-hrms-cron-2026)

7. API Route Security & Schema Sync

All API routes that access the Employee table include inline schema-sync statements at the start of each handler. This ensures ALL columns from the Prisma schema exist in the actual database table before any query runs. This prevents P2022 "column does not exist" errors on tenant databases that haven't been fully migrated.

7.1 Routes with Inline Schema-Sync

Route	Method	Schema-Sync
/api/employees	GET	30+ ALTER TABLE statements
/api/employees	POST	30+ ALTER TABLE statements
/api/employees/[id]	GET	30+ ALTER TABLE statements
/api/employees/[id]	PUT	7 ALTER TABLE statements
/api/employees/credentials	GET	37 ALTER TABLE statements (Employee + User)
/api/users/profile	GET	30+ ALTER TABLE statements (both DBs)
/api/employees/probation	GET	CREATE TABLE IF NOT EXISTS PerformanceReview
/api/employees/probation	POST	CREATE TABLE IF NOT EXISTS PerformanceReview

7.2 Build-Time Schema Sync

In addition to inline schema-sync, the following scripts run at Vercel build time to ensure all columns exist before the app starts:

Script	When	What
scripts/_init-table-fixes.js	Build time (before next build)	ALTER TABLE ADD COLUMN for all Employee fields
scripts/schema-sync.js	Build time (before next build)	Full schema sync including FK constraints
/api/db-push (POST)	Request time (on first page load)	Build-time schema fix as fallback

8. Official Email Validation

Only official/company email addresses are allowed for login. Personal email addresses (gmail, yahoo, outlook, etc.) are blocked at the login gate. Candidates are exempt and can use personal emails via the candidate portal.

8.1 Blocked Personal Email Domains

Blocked Domain
gmail.com
googlemail.com
outlook.com
hotmail.com
live.com
yahoo.com
yahoo.in
icloud.com
protonmail.com
rediffmail.com
aol.com
zoho.com
mail.com
tempmail.com
+ 15 more

The validation is applied in both `/api/auth/login` (login gate) and `/api/employees POST` (employee creation). The `validateOfficialEmail()` function in `src/lib/validators.ts` checks the email domain against the blocked list and returns a clear error message if a personal email is used.

9. Super Admin Capabilities

The Super Admin dashboard (`3boxeshrms.com`) provides platform-level management capabilities. Super admin can view all tenants, manage subscriptions, and perform administrative actions on individual tenants.

Feature	Description
Register Separate DBs	Registers TenantDatabase records for demo and tenant DBs
Seed DB	Seeds the dedicated tenant database with companies, employees, etc.
Clear Prob	Clears all probation review data for a tenant

Feature	Description
Reset Pwd	Resets ALL user passwords to MarqAI@2026 (tenant + platform DB)
Purge Dups	Deletes duplicate employees and syncs passwords across DBs
Backup	Creates a backup of the tenant database
Reset Employees	Deletes all employees except admin user
Tenant Status	Activate/Suspend/Pending approval for tenants

10. Confirmation & Compliance Checklist

CONFIRMED: The Super Admin database and Tenant databases are SEPARATE. Tenant data will NOT fall back to the platform/super admin DB for any data.

#	Check	Status
1	Platform DB (neondb) stores ONLY: Tenant, TenantDatabase, User (super_admin), TenantRegistration, Subscription	CONFIRMED
2	Tenant DBs store: Company, Department, Employee, Payroll, Attendance, Leave, etc	CONFIRMED
3	NO tenant data is read from or written to the platform DB	CONFIRMED
4	Super admin fetches tenant data via getDbForTenant(slug) when viewing a specific tenant	CONFIRMED
5	Platform DB fallbacks for employee data removed from /api/users/profile GET	CONFIRMED
6	Platform DB fallbacks for employee data removed from /api/users/profile PATCH	CONFIRMED
7	Type 1 (super admin tenant resolution via Company -> Group -> Tenant -> getDbForTenant(slug))	CONFIRMED
8	Official email only for login (no personal emails)	CONFIRMED
9	Employee Status: Confirmed, Probation, Contract, Internship, Consultant	CONFIRMED
10	Employee Type: Full Time, Part Time, Contract, Temporary, Consultant	CONFIRMED
11	Probation auto-creates review with start=dateOfJoining, end=editable	CONFIRMED
12	2-level workflow: HR review -> MD/Admin final confirmation	CONFIRMED
13	10-day email/notification alerts via cron endpoint	CONFIRMED
14	Inline schema-sync on all Employee API routes	CONFIRMED
15	Red asterisks (*) on all mandatory form fields	CONFIRMED

This document confirms that the 3Boxes HRMS architecture enforces complete database separation between the Super Admin platform and individual tenants. The Golden Rules are permanently enforced through API-level guards, inline schema-sync, and the removal of all platform DB fallbacks for tenant data.